

Versi 3.0
September 2026



PEMROGRAMAN WEB

MODUL 2 - JAVASCRIPT FUNDAMENTALS & DOM MANIPULATION

Author:

Christian Sri Kusuma Aditya, S.Kom., M.Kom
Nisrina Nurhafizhah
Shofwan Alif Alfani

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	4
TUJUAN.....	4
TARGET MODUL.....	4
PERSIAPAN.....	4
KATA KUNCI.....	4
MATERI.....	5
1. Peran JavaScript dalam Pengembangan Web.....	5
1.1. Cara Menulis JavaScript pada Halaman Web.....	5
1.2. Console pada Browser.....	7
2. Dasar JavaScript.....	7
2.1. Variabel.....	7
2.2. Function.....	8
3. ES6+ Features.....	9
3.1. Arrow Function.....	9
3.2. Template Literal.....	10
3.3. Destructuring.....	10
3.4. Spread dan Rest.....	11
3.5. Modules: Import dan Export.....	12
4. Array dan Array Methods.....	13
4.1. Array of Object.....	13
4.2. Array Methods.....	14
5. DOM Manipulation.....	16
5.1. Memilih dan Mengubah Elemen.....	16
5.2. Class, Attribute, dan Elemen Dinamis.....	17
6. Event Handling.....	18
7. Asynchronous JavaScript.....	20
7.1. Synchronous vs Asynchronous.....	20
7.2. Callback, Promise, dan Async/Await.....	21
8. Konsep REST API.....	22
8.1. Istilah Penting dalam REST API.....	23
8.2. HTTP Method.....	23
8.3. JSON.....	24
8.4. Menguji API dengan Postman.....	25
8.5. Membuat Postman Collection.....	26
Membuat Collection.....	26
Membuat Folder.....	27
Membuat Request.....	27
8.6. HTTP Status Code.....	27



9. Fetch API.....	28
9.1. Alur Fetch API.....	28
9.2. Fetch untuk Operasi CRUD.....	30
9.3. Penanganan Error pada Fetch API.....	31
CODELAB.....	33
DEMO.....	34
RUBRIK PENILAIAN.....	35



PENDAHULUAN

TUJUAN

1. Memberikan pemahaman kepada mahasiswa mengenai konsep dasar JavaScript dan perannya dalam pengembangan web modern
2. Membekali mahasiswa dengan kemampuan memanipulasi DOM, menangani event, dan menggunakan fitur JavaScript modern (ES6+)
3. Memberikan pemahaman mengenai konsep asynchronous JavaScript, REST API, dan penggunaan Fetch API untuk berkomunikasi dengan server

TARGET MODUL

1. Mahasiswa mampu menulis dan mengorganisir kode JavaScript menggunakan konsep dasar hingga fitur ES6+ dengan baik
2. Mahasiswa mampu membangun interaksi halaman web menggunakan DOM Manipulation, Event Handling, dan mengambil data dari REST API menggunakan Fetch API

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop or personal computer
3. Interactive Python Notebook (.ipynb)
4. Strong motivation and commitment to learning

KATA KUNCI

JavaScript, DOM Manipulation, Event Handling, ES6+, Asynchronous JavaScript, REST API, Fetch API



MATERI

1. Peran JavaScript dalam Pengembangan Web

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat halaman web lebih interaktif. Jika HTML digunakan untuk membuat struktur halaman dan CSS digunakan untuk mengatur tampilan, maka JavaScript digunakan untuk menambahkan logika dan perilaku pada halaman.

Tabel 1.1: Perbandingan fungsi HTML, CSS, dan JavaScript beserta contoh penggunaan

Teknologi	Fungsi Utama	Contoh Penggunaan
HTML	Membuat struktur halaman	Heading, paragraf, gambar, form, button
CSS	Mengatur tampilan halaman	Warna, ukuran teks, layout, responsive design
JavaScript	Menambahkan interaksi dan logika	Klik tombol, validasi form, filter data, fetch API

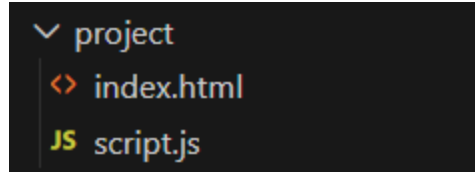
1.1. Cara Menulis JavaScript pada Halaman Web

Tabel 1.2: Cara penulisan JavaScript beserta contoh sintaks

Cara Penulisan	Penjelasan	Contoh Sintaks
Internal JavaScript	Ditulis langsung di dalam file HTML menggunakan tag <code><script></code>	<code><script>console.log("Halo");</script></code>
External JavaScript	Ditulis di file terpisah dengan ekstensi <code>.js</code>	<code><script src="script.js"></script></code>
External dengan <code>defer</code>	File JavaScript dijalankan setelah HTML selesai dibaca	<code><script src="script.js" defer></script></code>

External JavaScript lebih direkomendasikan untuk proyek karena kode HTML dan JavaScript menjadi lebih terpisah. Dengan cara ini, file menjadi lebih rapi dan lebih mudah dikelola.



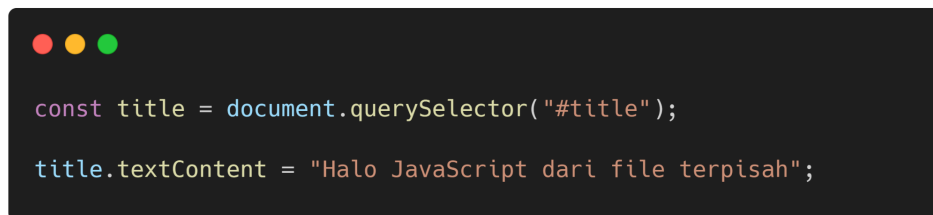


Gambar 1.1: Contoh struktur folder project (HTML dan JS terpisah)



Gambar 1.2: Contoh isi file index.html dengan tag script

Catatan: Posisi tag <script> harus diperhatikan. JavaScript sering mengambil elemen HTML. Jika JavaScript dijalankan sebelum elemen HTML dibaca oleh browser, elemen tersebut belum dapat ditemukan dan berpotensi menyebabkan error.



Gambar 1.3: Contoh isi file script.js



1.2. Console pada Browser

Saat belajar JavaScript, console di browser sering digunakan. Console membantu kita melihat hasil output, mengecek isi variabel, dan membaca pesan error.

Tabel 1.3: Sintaks console (log, warn, error) dan fungsinya

Kebutuhan	Sintaks	Fungsi
Menampilkan output biasa	<code>console.log("Belajar JavaScript");</code>	Menampilkan pesan atau nilai
Menampilkan peringatan	<code>console.warn("Data belum lengkap");</code>	Memberi pesan peringatan
Menampilkan error	<code>console.error("Terjadi error");</code>	Menampilkan pesan error

```
> const message = "Belajar JavaScript";
    console.log(message);
```

Gambar 1.4: Contoh penulisan console.log()

```
Belajar JavaScript VM135:3
< undefined
```

Gambar 1.5: Tampilan output di console browser

Untuk membuka console:

1. Klik kanan pada halaman web.
2. Pilih **Inspect** atau **Inspect Element**.
3. Buka tab **Console**.

`console.log()` sangat berguna saat belajar dan melakukan debugging. Namun, untuk website yang sudah siap digunakan oleh user, log yang tidak diperlukan sebaiknya dihapus.

2. Dasar JavaScript

2.1. Variabel

Variabel digunakan untuk menyimpan data. Dalam JavaScript modern, variabel biasanya dibuat menggunakan `const` atau `let`.



Tabel 2.1: Keyword variabel (*const*, *let*, *var*), fungsi, contoh, dan catatan pemakaian

Keyword	Fungsi	Contoh	Catatan
<code>const</code>	Menyimpan nilai yang tidak akan diganti dengan assignment baru	<code>const appName = "Web App";</code>	Gunakan sebagai pilihan utama
<code>let</code>	Menyimpan nilai yang bisa berubah	<code>let counter = 0;</code>	Cocok untuk nilai yang berubah
<code>var</code>	Cara lama membuat variabel	<code>var name = "Nisrin";</code>	Sebaiknya dihindari pada JavaScript modern

```

const appName = "Website Pembelajaran";
let totalClick = 0;

totalClick = totalClick + 1;

```

Gambar 2.1: Contoh deklarasi variabel *const*, *let*

2.2. Function

Function digunakan untuk menyimpan blok kode agar dapat digunakan kembali. Dalam pengembangan web, function sering digunakan untuk:

- Validasi form
- Render data
- Mengambil data API
- Memisahkan logika agar kode lebih rapi

Tabel 2.2: Bentuk function (*declaration* vs *expression*) beserta contoh dan catatan

Bentuk Function	Contoh	Catatan
Function declaration	<code>function getData() {}</code>	Mudah dipahami dan umum digunakan
Function expression	<code>const getData = function () {};</code>	Function disimpan dalam variabel



```
function getResult(score) {
  return score >= 75 ? "Lulus" : "Belum Lulus";
}
```

Gambar 2.2: Contoh function declaration

```
const getStatus = function (isActive) {
  return isActive ? "Aktif" : "Tidak Aktif";
};
```

Gambar 2.3: Contoh function expression

3. ES6+ Features

ES6+ adalah kumpulan fitur JavaScript modern yang membuat kode lebih ringkas, rapi, dan mudah dibaca. Fitur ini banyak digunakan dalam pengembangan frontend saat ini.

3.1. Arrow Function

Arrow function adalah cara menulis function dengan bentuk yang lebih singkat. Bentuk ini sering digunakan dalam JavaScript modern, terutama untuk callback pada metode array, penanganan event, dan Fetch API.

Tabel 3.1: Perbandingan function biasa vs arrow function

Bentuk	Contoh
Function biasa	<code>function multiply(number) { return number * 2; }</code>
Arrow function	<code>const multiply = (number) => number * 2;</code>



3.2. Template Literal

Template literal digunakan untuk membuat string dengan lebih fleksibel. Template literal menggunakan backtick dan dapat menyisipkan variabel dengan `${}`.

Tabel 3.2: Perbandingan string biasa vs template literal

String Biasa	Template Literal
Menggunakan " " atau ' '	Menggunakan backtick
Perlu operator +	Bisa menggunakan <code>\${}</code>
Kurang nyaman untuk HTML panjang	Lebih mudah untuk HTML dinamis

Template literal juga sangat berguna untuk membuat HTML yang dinamis.

```
const title = "Belajar JavaScript";
const category = "Programming";

const articleCard = `
  <div>
    <h3>${title}</h3>
    <p>${category}</p>
  </div>
`;
```

Gambar 3.1: Contoh template literal untuk HTML dinamis

3.3. Destructuring

Destructuring digunakan untuk mengambil nilai dari objek atau array dengan cara yang lebih ringkas.

Tabel 3.3: Perbandingan object destructuring vs array destructuring

Object Destructuring	Array Destructuring
Berdasarkan nama properti	Berdasarkan urutan indeks
<code>{ name, email } = user</code>	<code>[first, second] = colors</code>
Cocok untuk data object/API	Cocok untuk array sederhana



```
const user = {
  name: "Rani",
  email: "rani@mail.com",
  role: "student"
};
```

Gambar 3.2: Contoh object

```
const name = user.name;
const email = user.email;
```

Gambar 3.3: Contoh cara biasa mengambil properti object (tanpa destructuring)

```
const { name, email } = user;
```

Gambar 3.4: Contoh penggunaan object destructuring

```
const colors = ["red", "green", "blue"];
const [firstColor, secondColor] = colors;
```

Gambar 3.5: Contoh array destructuring

3.4. Spread dan Rest

Spread dan rest sama-sama menggunakan tanda tiga titik. Namun, fungsinya berbeda-beda tergantung pada cara penggunaannya.

Tabel 3.4: Perbandingan spread vs rest

Spread	Rest
Menyebarkan isi data	Mengumpulkan data
Dipakai saat membuat array/object baru	Dipakai pada parameter function
<code>const b = [...a]</code>	<code>function test(...args)</code>



```
const frontendTopics = ["HTML", "CSS"];
const allTopics = [...frontendTopics, "JavaScript"];

console.log(allTopics);
```

Gambar 3.6: Contoh spread pada array

```
const showTopics = (...topics) => {
  console.log(topics);
};

showTopics("HTML", "CSS", "JavaScript");
```

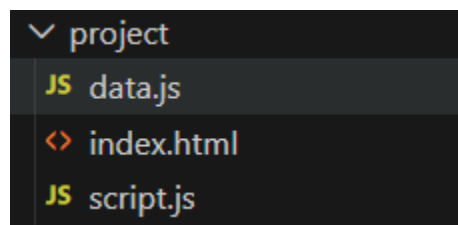
Gambar 3.7: Contoh rest pada parameter function

```
["HTML", "CSS", "JavaScript"]
```

Gambar 3.8: Output hasil contoh rest

3.5. Modules: Import dan Export

Module digunakan untuk memisahkan kode JavaScript ke dalam beberapa file. Tujuannya agar proyek lebih rapi, terutama jika kode semakin banyak.



Gambar 3.9: Struktur folder untuk contoh modul



```
export const topics = ["HTML", "CSS", "JavaScript"];
```

Gambar 3.10: Contoh isi file data.js (export)

```
import { topics } from "../data.js";  
console.log(topics);
```

Gambar 3.11: Contoh isi file script.js (import)

Agar browser mengenali sistem modul, skrip HTML perlu ditulis dengan `type="module"`. Tanpa `type="module"`, browser tidak akan mengenali sintaks `import` dan `export`.

```
<script type="module" src="script.js"></script>
```

Gambar 3.12: Contoh penulisan script `type="module"` di HTML

4. Array dan Array Methods

Array digunakan untuk menyimpan banyak data dalam satu variabel. Dalam pengembangan web, array sering digunakan untuk menyimpan daftar data, seperti artikel, produk, pengguna, komentar, atau data dari API. Array method penting karena data dari API biasanya berbentuk array of objects. Data tersebut perlu diolah terlebih dahulu sebelum ditampilkan di halaman.

4.1. Array of Object

Array of objects adalah array yang berisi beberapa objek.

```
const products = [  
  { id: 1, name: "Laptop", category: "Elektronik", price: 7000000 },  
  { id: 2, name: "Mouse", category: "Aksesoris", price: 100000 },  
  { id: 3, name: "Keyboard", category: "Aksesoris", price: 250000 }  
];
```

Gambar 4.1: Contoh array of objects



```
console.log(products[0].name);
console.log(products[1].price);
```

Gambar 4.2: Contoh cara membaca data array of objects

```
Laptop
100000
```

Gambar 4.3: Output hasil membaca data array of objects

4.2. Array Methods

Tabel 4.1: Array method : fungsi, hasil, contoh sintaks

Method	Fungsi	Hasil	Contoh Sintaks
<code>forEach()</code>	Menjalankan aksi untuk setiap item	Tidak membuat array baru	<code>products.forEach(p => console.log(p.name));</code>
<code>map()</code>	Mengubah data	Array baru	<code>products.map(p => p.name);</code>
<code>filter()</code>	Menyaring data	Array baru	<code>products.filter(p => p.category === "Aksesoris");</code>
<code>find()</code>	Mencari satu data	Satu item / <code>undefined</code>	<code>products.find(p => p.id === 2);</code>
<code>reduce()</code>	Menghasilkan satu nilai akhir	Satu nilai	<code>products.reduce((total, p) => total + p.price, 0);</code>
<code>join()</code>	Menggabungkan array menjadi string	String	<code>htmlItems.join("");</code>



```
// forEach: menjalankan aksi untuk setiap item
products.forEach((product) => {
  console.log(product.name);
});
```

```
// map: membuat array baru berisi nama produk
const productNames = products.map((product) => product.name);
```

```
// filter: menyaring produk kategori Aksesoris
const accessories = products.filter((product) => {
  return product.category === "Aksesoris";
});
```

```
// find: mencari satu produk berdasarkan id
const selectedProduct = products.find((product) => {
  return product.id === 2;
});
```

```
// reduce: menghitung total harga semua produk
const totalPrice = products.reduce((total, product) => {
  return total + product.price;
}, 0);
```

```
// map + template literal + join: membuat HTML dari data
const productHTML = products
  .map((product) => {
    return `
      <div>
        <h3>${product.name}</h3>
        <p>Harga: Rp${product.price}</p>
      </div>
    `;
  })
  .join("");
```

Gambar 4.4–4.9 : Rangkaian contoh penggunaan array method



5. DOM Manipulation

DOM atau Document Object Model adalah representasi struktur HTML yang dapat diakses oleh JavaScript. Dengan DOM, JavaScript bisa:

- Memilih elemen HTML
- Mengubah teks
- Mengubah class
- Mengambil nilai input
- Menambahkan elemen baru
- Menampilkan data ke halaman

JavaScript tidak mengubah file HTML asli secara permanen. JavaScript hanya mengubah tampilan halaman yang sedang berjalan di browser. Jika halaman di-refresh, perubahan dapat kembali ke kondisi semula.

5.1. Memilih dan Mengubah Elemen

Sebelum mengubah elemen, JavaScript perlu memilih elemen tersebut terlebih dahulu.

Tabel 5.1: Method/property DOM

Method/Property	Fungsi	Contoh
<code>querySelector()</code>	Memilih satu elemen pertama	<code>document.querySelector("#title")</code>
<code>querySelectorAll()</code>	Memilih banyak elemen	<code>document.querySelectorAll(".btn")</code>
<code>getElementById()</code>	Memilih elemen berdasarkan id	<code>document.getElementById("title")</code>
<code>textContent</code>	Mengubah teks biasa	<code>title.textContent = "Halo JavaScript";</code>
<code>innerHTML</code>	Mengubah isi HTML	<code>box.innerHTML = "<p>Data berhasil dimuat</p>";</code>
<code>value</code>	Mengambil atau mengubah nilai input	<code>input.value</code>

```
const title = document.querySelector("#title");
title.textContent = "Belajar DOM Manipulation";
```

Gambar 5.1 : Contoh penggunaan querySelector



5.2. Class, Attribute, dan Elemen Dinamis

Class pada elemen HTML dapat diubah menggunakan `classList`. Fitur ini sering digunakan untuk mengatur tampilan, misalnya menampilkan menu, menyembunyikan form, atau memberikan gaya aktif pada tombol. JavaScript juga dapat mengubah atribut seperti `src`, `href`, `alt`, dan `disabled`. JavaScript juga dapat digunakan untuk membuat elemen baru secara dinamis.

Tabel 5.2: Method mengubah class, attribute, dan membuat elemen dinamis (`classList`, `setAttribute`, `createElement`, `dst.`)

Method/Cara	Fungsi	Contoh Sintaks
Mengubah Class		
<code>classList.add()</code>	Menambahkan class	<code>box.classList.add("active");</code>
<code>classList.remove()</code>	Menghapus class	<code>box.classList.remove("active");</code>
<code>classList.toggle()</code>	Menambah atau menghapus class	<code>box.classList.toggle("hidden");</code>
<code>classList.contains()</code>	Mengecek apakah class ada	<code>box.classList.contains("active");</code>
Mengubah Attribute		
<code>setAttribute()</code>	Mengubah attribute	<code>image.setAttribute("src", "new.jpg")</code>
<code>getAttribute()</code>	Mengambil nilai attribute	<code>image.getAttribute("src")</code>
<code>removeAttribute()</code>	Menghapus attribute	<code>button.removeAttribute("disabled")</code>
Membuat Elemen Dinamis		
<code>innerHTML</code>	Praktis untuk menampilkan HTML dari data	<code>container.innerHTML = "<p>Halo</p>";</code>



<code>createElement()</code>	Lebih aman dan terstruktur untuk membuat elemen satu per satu	<code>const item = document.createElement("li");</code>
<code>appendChild()</code>	Menambahkan elemen ke dalam elemen lain	<code>list.appendChild(item);</code>

```

<h1 id="title">Daftar Materi</h1>
<input id="search-input" type="text">
<ul id="topic-list"></ul>

```

Gambar 5.2: Contoh struktur elemen HTML untuk manipulasi DOM

```

const title = document.querySelector("#title");
const input = document.querySelector("#search-input");
const list = document.querySelector("#topic-list");

const topics = ["HTML", "CSS", "JavaScript"];

title.textContent = "Materi Pemrograman Web";
input.setAttribute("placeholder", "Cari materi");

const topicHTML = topics
  .map(topic => `<li>${topic}</li>`)
  .join("");

list.innerHTML = topicHTML;

```

Gambar 5.3 : Contoh manipulasi elemen DOM menggunakan JavaScript

6. Event Handling

Event handling adalah cara JavaScript menangani aksi dari user, seperti:

- Klik tombol
- Mengetik di input
- Memilih dropdown
- Mengirim form



Tabel 6.1: Jenis event (*click, input, submit, change, keydown*) dan contoh penggunaannya

Event	Kapan Terjadi	Contoh Penggunaan
click	Saat elemen diklik	Tombol menu, tombol submit, tombol detail
input	Saat user mengetik	Search/filter real-time
submit	Saat form dikirim	Validasi form
change	Saat nilai input/select berubah	Dropdown kategori, checkbox
keydown	Saat keyboard ditekan	Shortcut, pencarian dengan Enter

```

element.addEventListener("namaEvent", () => {
  // kode yang dijalankan
});

```

Gambar 6.1: Format umum penulisan event listener

```

<form id="user-form">
  <input id="name-input" type="text" placeholder="Masukkan nama">
  <button type="submit">Simpan</button>
</form>

<p id="message"></p>

```

Gambar 6.2 : Contoh struktur form untuk event handling



```

const form = document.querySelector("#user-form");
const nameInput = document.querySelector("#name-input");
const message = document.querySelector("#message");

nameInput.addEventListener("input", function () {
  message.textContent = `Nama: ${nameInput.value}`;
});

form.addEventListener("submit", function (event) {
  event.preventDefault();

  if (nameInput.value.trim() === "") {
    message.textContent = "Nama wajib diisi";
    return;
  }

  message.textContent = "Data berhasil disimpan";
});

```

Gambar 6.3: Contoh event handling pada input dan submit form

7. Asynchronous JavaScript

Asynchronous JavaScript digunakan ketika ada proses yang membutuhkan waktu, misalnya:

- Mengambil data dari API
- Membaca file
- Menunggu respons dari server

Jika semua kode harus menunggu proses selesai, halaman bisa terasa lambat. Dengan JavaScript, proses yang membutuhkan waktu dapat berjalan tanpa menghentikan seluruh program.

7.1. Synchronous vs Asynchronous

Synchronous berarti kode berjalan berurutan.

```

console.log("Satu");
console.log("Dua");
console.log("Tiga");

```

Gambar 7.1: Contoh kode synchronous



```
Satu
Dua
Tiga
```

Gambar 7.2: Output kode synchronous

Asynchronous berarti ada proses yang bisa berjalan sambil menunggu hasil, tanpa menghentikan seluruh kode lainnya.

```
console.log("Satu");

setTimeout(() => {
  console.log("Dua");
}, 2000);

console.log("Tiga");
```

Gambar 7.3: Contoh kode asynchronous (setTimeout)

```
Satu
Tiga
Dua
```

Gambar 7.4: Output kode asynchronous

Kode "Dua" muncul terakhir karena dijalankan setelah 2 detik. Sementara itu, JavaScript tetap melanjutkan ke baris berikutnya tanpa menunggu `setTimeout()` selesai.

7.2. Callback, Promise, dan Async/Await

Tabel 7.1: Konsep Callback, Promise, dan Async/Await beserta penjelasannya

Konsep	Penjelasan
Callback	Function yang dijalankan setelah proses tertentu
Promise	Object yang mewakili proses async yang bisa berhasil/gagal
Async/Await	Cara menulis Promise agar lebih mudah dibaca



```

console.log("Mulai");

// asynchronous dengan setTimeout
setTimeout(() => {
  console.log("Proses async selesai");
}, 2000);

// Promise
const getMessage = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Promise berhasil");
  } else {
    reject("Promise gagal");
  }
});

// async/await
const showMessage = async () => {
  try {
    const message = await getMessage;
    console.log(message);
  } catch (error) {
    console.log(error);
  }
};

showMessage();

console.log("Selesai");

```

Gambar 7.5: Contoh Callback, Promise, dan Async/Await

```

Mulai
Selesai
Promise berhasil
Proses async selesai

```

Gambar 7.6: Contoh Output Callback, Promise, dan Async/Await

8. Konsep REST API

REST API adalah cara client dan server berkomunikasi melalui protokol HTTP. Dalam konteks frontend, browser bertindak sebagai client yang meminta data, sedangkan server menyediakan data tersebut.



8.1. Istilah Penting dalam REST API

Tabel 8.1: Istilah penting REST API

Istilah	Penjelasan	Contoh
Client	Pihak yang meminta data	Browser
Server	Pihak yang menyediakan data	API server
API	Penghubung client dan server	REST API
Endpoint	URL untuk mengakses data tertentu	<code>/users, /posts</code>
Request	Permintaan dari client ke server	Mengambil data
Response	Balasan dari server	Data JSON

Contoh endpoint public API:

<https://jsonplaceholder.typicode.com/users>

Endpoint tersebut dapat digunakan untuk mengambil data pengguna dari JSONPlaceholder.

8.2. HTTP Method

HTTP method menunjukkan jenis aksi yang ingin dilakukan pada data.

Tabel 8.2: HTTP method (GET, POST, PUT, PATCH, DELETE) dan contoh kasus

Method	Fungsi	Contoh Kasus
GET	Mengambil data	Menampilkan daftar produk
POST	Mengirim data baru	Menambahkan komentar
PUT	Mengganti seluruh data	Mengubah profil lengkap
PATCH	Mengubah sebagian data	Mengubah nama saja
DELETE	Menghapus data	Menghapus item



8.3. JSON

JSON atau JavaScript Object Notation adalah format data yang sering digunakan dalam API. Bentuknya mirip dengan objek dan array di JavaScript, tetapi memiliki aturan yang lebih ketat.

```
[
  {
    "id": 1,
    "name": "Shofwan Alif",
    "email": "alif@mail.com"
  },
  {
    "id": 2,
    "name": "Nisrina Nuha",
    "email": "nisrinnuha@mail.com"
  }
]
```

Gambar 8.1 : Contoh array JSON

Tabel 8.3: Perbedaan JSON dan object JavaScript

JSON	Object JavaScript
Format pertukaran data	Struktur data di JavaScript
Key wajib menggunakan kutip dua	Key tidak selalu wajib menggunakan kutip
Tidak bisa berisi function	Bisa berisi function
Biasanya berasal dari API	Digunakan langsung di kode JavaScript

Data JSON yang diterima dari server biasanya masih berupa teks, sehingga perlu diubah menjadi objek atau array JavaScript agar dapat diolah. Proses ini akan dibahas lebih lanjut pada bab Fetch API.

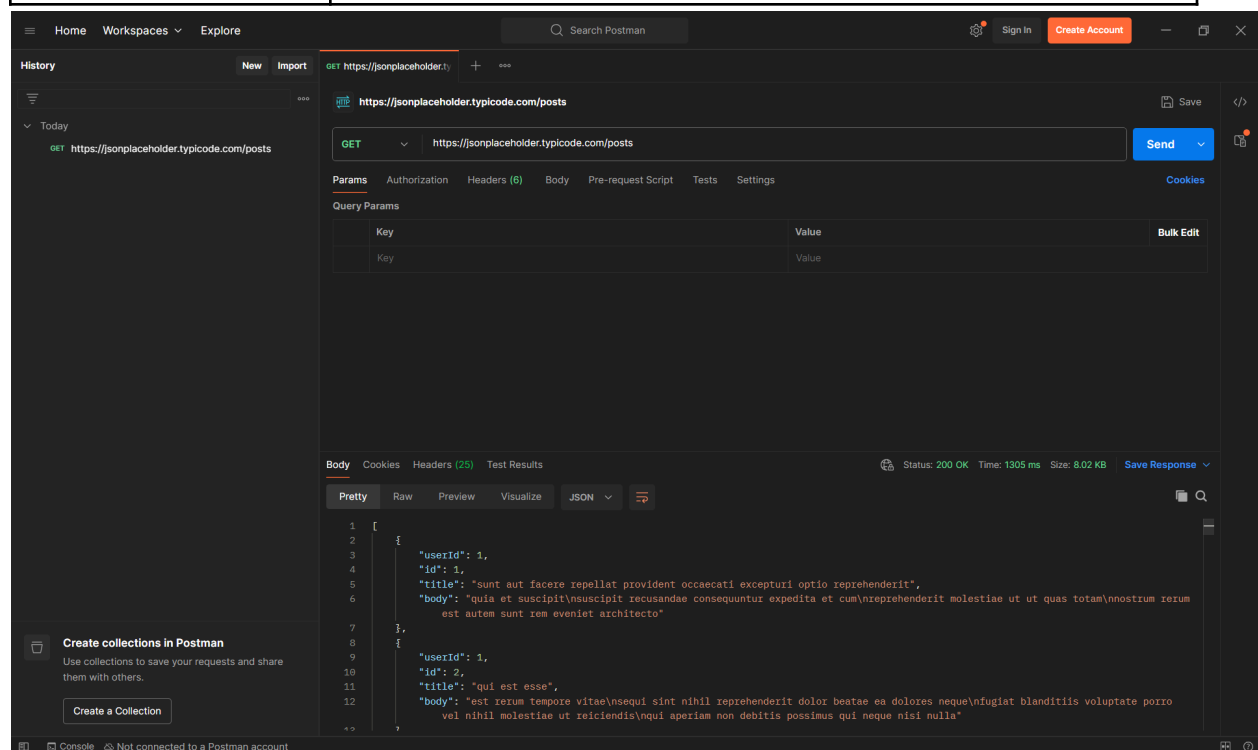


8.4. Menguji API dengan Postman

Sebelum mengikuti bagian ini, pastikan Postman sudah terinstal di perangkat masing-masing. Cara instalasi Postman telah dijelaskan pada Modul 0

Tabel 8.4: Langkah menguji API dengan Postman

Langkah	Keterangan
1. Buka Postman	Pastikan aplikasi Postman sudah terinstal dan dapat dijalankan
2. Pilih method	Misalnya GET , POST , PUT , atau DELETE
3. Masukkan endpoint	Contoh: https://jsonplaceholder.typicode.com/users
4. Klik Send	Postman akan mengirim request ke server
5. Lihat response	Periksa status code dan body response

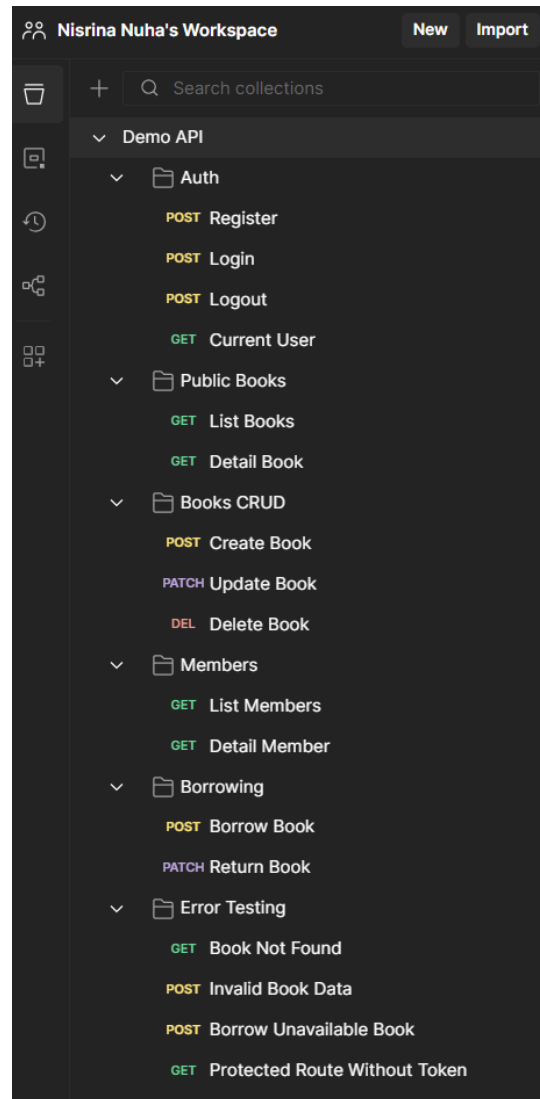


Gambar 8.2 : Tampilan pengujian API menggunakan Postman



8.5. Membuat Postman Collection

Agar request API tersusun rapi, buat satu Collection untuk menyimpan seluruh request dalam project. Request kemudian dikelompokkan ke dalam folder berdasarkan fitur atau jenis prosesnya. Pada contoh ini, Collection diberi nama: **Demo API**. Struktur Collection yang digunakan adalah:



Gambar 8.3 : Struktur Postman Collection berdasarkan fitur API

Membuat Collection

1. Buka aplikasi Postman.
2. Pilih menu **Collections**.
3. Klik **New Collection**.
4. Beri nama Collection



Membuat Folder

Buat folder di dalam Collection untuk mengelompokkan request.

1. Klik ikon tiga titik pada Collection.
2. Pilih **Add folder**.
3. Masukkan nama folder.
4. Ulangi sesuai kebutuhan.

Membuat Request

Untuk membuat request baru:

1. Klik ikon tiga titik pada folder tujuan.
2. Pilih **Add request**.
3. Beri nama request.
4. Pilih HTTP method, seperti **GET**, **POST**, **PATCH**, atau **DELETE**.
5. Masukkan URL endpoint.
6. Klik **Save**.
7. Pilih folder tempat permintaan tersebut disimpan.

Pengelompokan folder dapat disesuaikan dengan fitur pada proyek masing-masing. Gunakan nama request yang jelas agar fungsi setiap request mudah dikenali.

Catatan: Pengujian request, penggunaan body, header, token, dan protected route akan dibahas sesuai kebutuhan pada modul berikutnya. Pada bagian ini, fokus utamanya adalah membuat struktur Collection, folder, dan request.

8.6. HTTP Status Code

Status code adalah kode angka dari server untuk memberi tahu apakah request berhasil atau gagal.

Tabel 8.5: Daftar HTTP status code

Status Code	Arti
200	OK / request berhasil
201	Created / data berhasil dibuat
400	Bad Request / request tidak sesuai
401	Unauthorized / belum login atau token salah
403	Forbidden / tidak punya akses
404	Not Found / data atau endpoint tidak ditemukan
500	Server Error/error dari server



9. Fetch API

Fetch API digunakan untuk melakukan permintaan ke API menggunakan JavaScript. Dengan Fetch API, kita bisa:

- Mengambil data dari server
- Mengirim data baru
- Mengubah data
- Menghapus data

Pada modul ini, fokus utamanya adalah mengambil data menggunakan GET, lalu menampilkannya di halaman.

9.1. Alur Fetch API

Tabel 9.1: Alur Fetch API

Langkah	Penjelasan	Contoh Sintaks
1. Kirim request	Mengakses endpoint API	<code>fetch(url)</code>
2. Terima response	Server mengirim response	<code>const response = await fetch(url);</code>
3. Ubah ke JSON	Response diubah menjadi data JS	<code>const data = await response.json();</code>
4. Tampilkan data	Data dirender ke halaman	<code>container.innerHTML = html;</code>

```
const response = await fetch(url);
const data = await response.json();
```

Gambar 9.1 : Sintaks dasar `fetch()` dan `response.json()`

`fetch(url)` digunakan untuk mengirim request ke API. `response.json()` digunakan untuk mengubah respons menjadi data JavaScript agar dapat diproses dan ditampilkan di halaman.

```
<input id="search-input" type="text" placeholder="Cari user...">
<p id="loading-message"></p>
<p id="error-message"></p>
<div id="user-list"></div>
```

Gambar 9.2: Contoh penerapan fetch API sederhana



```

const searchInput = document.querySelector("#search-input");
const userList = document.querySelector("#user-list");
const loadingMessage = document.querySelector("#loading-message");
const errorMessage = document.querySelector("#error-message");

let usersData = [];

const renderUsers = (users) => {
  userList.innerHTML = users
    .map((user) => {
      return `
        <div>
          <h3>${user.name}</h3>
          <p>${user.email}</p>
        </div>
      `;
    })
    .join("");
};

const getUsers = async () => {
  try {
    loadingMessage.textContent = "Loading data...";
    errorMessage.textContent = "";

    const response = await
    fetch("https://jsonplaceholder.typicode.com/users");

    if (!response.ok) {
      throw new Error("Data gagal dimuat");
    }

    const users = await response.json();

    usersData = users;
    renderUsers(usersData);

    loadingMessage.textContent = "";
  } catch (error) {
    loadingMessage.textContent = "";
    errorMessage.textContent = "Data gagal dimuat. Coba lagi nanti.";
  }
};

searchInput.addEventListener("input", () => {
  const keyword = searchInput.value.toLowerCase();

  const filteredUsers = usersData.filter((user) => {
    return user.name.toLowerCase().includes(keyword);
  });

  renderUsers(filteredUsers);
});

getUsers();

```

Gambar 9.3: Contoh lengkap fetch API dengan render data ke halaman



Catatan penting

Contoh di atas menampilkan data dengan `container.innerHTML = html`. Ini aman selama data berasal dari API tepercaya (seperti Jikan API). Namun, jika data yang dirender berasal dari input pengguna lain yang menggunakan `innerHTML` secara langsung, hal itu dapat membuka celah XSS (Cross-Site Scripting). Jika user lain berhasil menyisipkan kode seperti `<script>` atau atribut `onerror` pada judul/deskripsi kontennya, kode tersebut dapat ikut dieksekusi di browser setiap orang yang melihat halaman tersebut.

```
// 1. Untuk teks mentah tanpa tag HTML: pakai textContent, bukan innerHTML
titleElement.textContent = data.title; // aman, tidak akan mengeksekusi
tag/script apa pun
// 2. Jika terpaksa pakai innerHTML untuk data dari user, escape dulu karakter
HTML-nya
function escapeHTML(text) {
  const div = document.createElement("div");
  div.textContent = text;
  return div.innerHTML;
}
container.innerHTML = `<p>${escapeHTML(data.title)}</p>`;
```

Gambar 9.4 : Contoh penggunaan `textContent` dan sanitasi HTML untuk mencegah XSS

Untuk data dari API publik tepercaya pada studi kasus modul ini, `innerHTML` tetap aman digunakan. Catatan ini penting untuk diingat kembali pada Modul 6, saat konten yang dirender berasal dari user lain, karena di situ celah XSS ini bisa langsung dipakai untuk menyerang token JWT yang tersimpan di `localStorage`

9.2. Fetch untuk Operasi CRUD

Tabel 9.2: Fetch API untuk operasi CRUD

Method	Fungsi	Sintaks Inti
GET	Mengambil data	<code>fetch(url)</code>
POST	Mengirim data baru	<code>fetch(url, { method: "POST", body: JSON.stringify(data) })</code>
PUT	Mengganti seluruh data	<code>fetch(url, { method: "PUT", body: JSON.stringify(data) })</code>
PATCH	Mengubah sebagian data	<code>fetch(url, { method: "PATCH", body: JSON.stringify(data) })</code>
DELETE	Menghapus data	<code>fetch(url, { method: "DELETE" })</code>



```
const newPost = {
  title: "Belajar Fetch API",
  body: "Mengirim data dengan POST",
  userId: 1
};

fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(newPost)
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log("Gagal mengirim data"));
```

Gambar 9.5: Contoh fetch API untuk operasi CRUD

9.3. Penanganan Error pada Fetch API

Fetch API tidak secara otomatis menganggap respons dengan status 404 atau 500 sebagai error. Promise dari `fetch()` hanya ditolak ketika terjadi masalah jaringan, seperti perangkat tidak terhubung ke internet, DNS gagal, koneksi terputus, atau permintaan diblokir oleh CORS. Jika server mengembalikan status 404 atau 500, proses `fetch()` tetap dianggap berhasil sehingga kode `.then()` atau `await` tetap berjalan. Oleh karena itu, `response.ok` perlu diperiksa secara manual. Contoh kode yang belum menangani error HTTP dengan benar:

```
// SALAH – tidak akan mendeteksi 404/500 sebagai error
try {
  const response = await fetch(url);
  const data = await response.json(); // tetap jalan meski status 404!
  renderData(data);
} catch (error) {
  showError(error.message); // baris ini TIDAK akan ke-trigger untuk error 404/500
}
```

Gambar 9.6 : Contoh fetch tanpa pengecekan `response.ok`



Kode tersebut belum cukup karena blok `catch` tidak akan dijalankan ketika server mengembalikan status `404` atau `500`. Untuk menangani error HTTP, gunakan `response.ok` sebelum mengubah response menjadi JSON:

```
// BENAR
async function ambilData(url) {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error(`Gagal mengambil data (status ${response.status})`);
    }
    const data = await response.json();
    renderData(data);
  } catch (error) {
    showError(error.message);
  }
}
```

Gambar 9.7 : Contoh fetch dengan pengecekan `response.ok`

Jika `response.ok` bernilai `false`, buat error dengan `throw new Error()`. Error tersebut kemudian akan ditangkap oleh blok `catch` dan dapat ditampilkan sebagai pesan error kepada pengguna. Contoh alur penanganan error:

1. `fetch()` mengirim request ke endpoint.
2. Program menerima respons dari server.
3. Program memeriksa `response.ok`
4. Jika `response.ok` bernilai `false`; program melempar error.
5. Blok `catch` menangkap error.
6. Pesan error ditampilkan kepada pengguna.

Pengecekan `response.ok` penting agar error seperti endpoint salah, data tidak ditemukan, atau server mengalami masalah tidak dianggap sebagai respons yang berhasil.



CODELAB

Membuat Interaksi pada Halaman Modul 1

Tambahkan interaksi sederhana pada halaman landing page yang sudah dibuat pada Modul 1 menggunakan JavaScript. Interaksi dapat berupa salah satu atau kombinasi berikut:

- Navbar toggle yang menampilkan dan menyembunyikan menu navigasi saat tombol menu diklik, terutama pada tampilan mobile
- Validasi form yang memberikan pesan peringatan jika input pada form belum diisi atau tidak sesuai format yang diharapkan

Mengambil dan Menampilkan Data dari Public API

Buat halaman baru yang mengambil data dari public API menggunakan Fetch API dengan method GET. Ketentuan:

- Tampilkan pesan loading saat data masih dalam proses pengambilan
- Tampilkan pesan error apabila proses pengambilan data gagal
- Render data yang berhasil diambil ke dalam bentuk card atau table sederhana di halaman



DEMO

Studi Kasus

Bangun halaman "The Vibe Grid", halaman utama FlipPlay yang menampilkan daftar media (anime/manga/drama) dalam bentuk grid card, diambil dari API publik (contoh: Jikan API <https://api.jikan.moe/v4/anime>).

Terapkan dua fitur berikut pada halaman tersebut:

- Unified Search: input pencarian yang memfilter card yang tampil berdasarkan judul secara real-time saat user mengetik.
- Aura-Based Discovery: sediakan beberapa tombol Aura (misalnya "High Energy", "Melancholy", "Binge Mode") yang, ketika diklik, akan menyaring data yang sudah diambil menjadi subset sesuai dengan Aura tersebut.

Tampilkan state loading saat data sedang diambil dan error message jika pengambilan data gagal.

Ketentuan Pengerjaan

Wajib menerapkan seluruh poin berikut pada website yang dibuat:

- Minimal dua event handling (misalnya klik, input, atau submit)
- Pengambilan data menggunakan Fetch API
- Render data ke dalam bentuk card, table, atau list
- Loading state saat data sedang diambil
- Error message apabila pengambilan data gagal
- Search atau filter sederhana pada data yang ditampilkan
- Penggunaan template literal untuk menyusun tampilan data secara dinamis
- Minimal satu array method (forEach, map, filter, find, atau reduce) dalam proses pengolahan data



RUBRIK PENILAIAN

Aspek Penilaian	Poin (%)
Codelab	Total 10%
Berhasil menerapkan interaksi pada halaman Modul 1 (navbar toggle/validasi form)	5
Berhasil mengambil & menampilkan data dari public API (loading & error state)	5
Demo	Total 80%
Event handling (minimal 2 jenis: klik/input/submit)	15
Fetch API & rendering data ke card/table/list	20
Loading state & error message	15
Search real-time & filter Aura-Based Discovery	15
Template literal & minimal satu array method	10
Kesesuaian dengan studi kasus The Vibe Grid	5
Pemahaman	Total 10%
Ketepatan menjawab pertanyaan	10

